EXPERIMENT - C

Name : Ved Khode PRN : 25070521176

6.1.1] Incremented Date

A) ALGORITHM

1. Start.
2. Read day.
3. Read month.
4. Read year.
5. If year ≤ 0, print "Invalid Date" and Stop.
6. If month < 1 or month > 12, print "Invalid Date" and Stop.
7. If month is 1, 3, 5, 7, 8, 10, 12 then max_days = 31.
8. Else if month is 4, 6, 9, 11 then max_days = 30.
9. Else if month is 2:

   If (year % 400 == 0) OR (year % 4 == 0 AND year % 100 ≠ 0)

   max_days = 29

   Else max_days =

   28
10. If day < 1 or day > max_days, print "Invalid Date" and Stop.
11. If day < max_days then day = day + 1.
12. Else

    day = 1

    If month == 12

    month = 1

    year = year +

    1

    Else

    month = month + 1
13. Print day-month-year in format DD-MM-YYYY.

EXPERIMENT - C

14. Stop.

B) PYTHON CODE

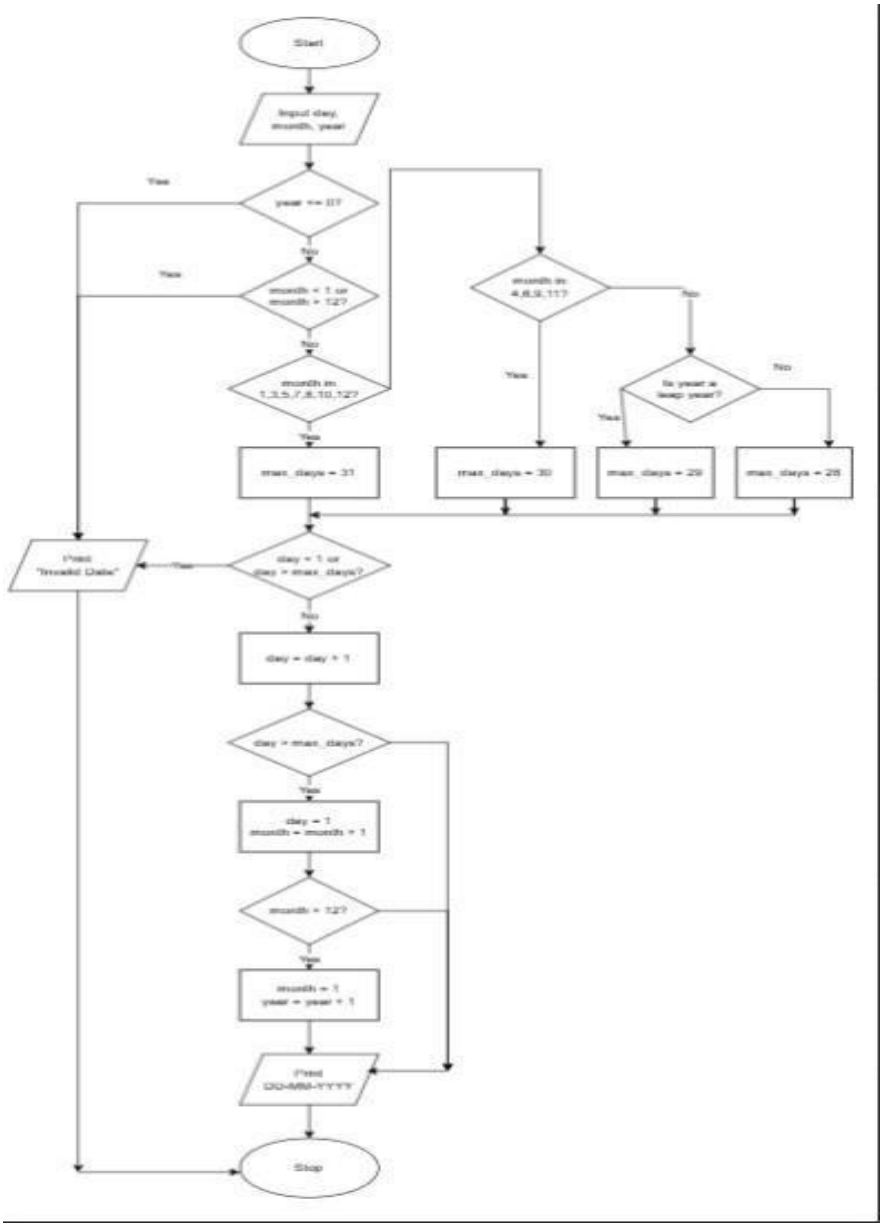
```
day = int(input())

month =

int(input())

year = int(input())



def is_leap(year):

  if (year % 400 == 0) or (year % 4 == 0 and year % 100 != 0):

    return True

  return False



def days_in_month(month, year):
 if month in [1, 3, 5, 7, 8, 10,
  12]:

    return 31

  elif month in [4, 6, 9, 11]:

    return 30

  elif month == 2:

    return 29 if is_leap(year) else 28

  else:

    return 0



if year <= 0 or month < 1 or month >
12:

  print("Invalid Date") elif day < 1 or day >

days_in_month(month, year):

  print("Invalid Date")

else:
```

EXPERIMENT - C

```
day += 1 if day >
days_in_month(month, year):
  day = 1 month
  += 1
  if month > 12:
    month = 1
    year += 1
    print(f"{day:02d}-{month:02d}-
{year}")
```

C) FLOWCHART

EXPERIMENT - C



EXPERIMENT - C

D) EXECUTION

CODETANTRA Home

ved.khode.batch2025@sitnagpur.siu.edu.in Support Logout

6.1.1. Incremented Date

02:30

Write a Python program to check if a given date is valid. If the date is valid, print the next day's date (incremented date). If the date is invalid, print "Invalid Date".

Date Validation Rules:

1. **Valid Month**: 1 to 12
2. **Valid Day**: Depends on the month and year
   - January (1), March (3), May (5), July (7), August (8), October (10), December (12): 1 to 31 days
   - April (4), June (6), September (9), November (11): 1 to 30 days
   - February (2): 1 to 29 days in a leap year and 1 to 28 days in a non-leap year
3. **Valid Year**: Any positive integer greater than zero.

Date Increment Rules:

If the date is valid, increment it by one day:

- If it's the last day of the month, move to the 1st day of the next month
- If it's December 31st, move to January 1st of the next year

Input Format:

- First line contains an integer representing the day
- Second line contains an integer representing the month
- Third line contains an integer representing the year

Output Format:

- If the date is valid, print the incremented date in the format:

```
<DD>-<MM>-<YYYY>
```

- If the date is invalid, print: "Invalid Date".

Sample Test Cases +

Explorer

nextDate.py

Submit

```python

day = int(input())
month = int(input())
year = int(input())

def is_leap_year(year):
    if (year % 400 == 0) or (year % 4 == 0 and year % 100 != 0):
        return True
    return False


def days_in_month(month, year):
    if month in [1, 3, 5, 7, 8, 10, 12]:
        return 31
    elif month in [4, 6, 9, 11]:
        return 30
    elif month == 2:
        if is_leap_year(year):
            return 29
        else:
```

Debugger

Average time	Maximum time
0.008 s	0.011 s
8.00 ms	11.00 ms

5 out of 5 shown test case(s) passed

5 out of 5 hidden test case(s) passed

Test case 1 8 ms Debug

Expected output	Actual output
15	15
3	3
2024	2024
16-03-2024	16-03-2024

Test case 2 41 ms

Terminal Test cases

‹ Prev Reset Submit Next ›